

Teach Module 6 README

Module 6: Streams And Functional Programming

This README captures the Module 6 teaching notes and practice lab. The course document was used only to identify the Module 6 topic. The explanations below are original teaching notes.

Core Idea

Module 6 is about writing cleaner Java code for processing collections using lambdas and streams.

Instead of writing every step manually:

```
for (Employee e : employees) {  
    if (e.getSalary() > 80000) {  
        names.add(e.getName());  
    }  
}
```

You can express the data-processing pipeline:

```
List<String> names = employees.stream()  
    .filter(e -> e.getSalary() > 80000)  
    .map(Employee::getName)  
    .toList();
```

The flow is:

```
source -> filter -> transform -> collect/result
```

Functional Programming In Java

Functional programming means writing code using small operations that describe what you want, instead of spelling out every step of how to do it.

Java supports functional-style programming through:

- Lambda expressions
- Functional interfaces
- Method references
- Streams
- Operations such as `filter` , `map` , `reduce` , and `collect`

Example lambda:

```
name -> name.toUpperCase()
```

This takes `name` and returns `name.toUpperCase()` .

Lambda Expressions

A lambda is a short function that can be passed around.

Basic syntax:

```
parameter -> expression
```

Examples:

```
x -> x * 2
name -> name.length()
user -> user.isActive()
```

With multiple parameters:

```
(a, b) -> a + b
```

With a block body:

```
name -> {
    System.out.println(name);
    return name.length();
}
```

Lambdas are commonly used when Java expects a functional interface.

Example:

```
@FunctionalInterface
interface Greeting {
    void sayHello(String name);
}
```

Usage:

```
Greeting greeting = name -> System.out.println("Hello " + name);
greeting.sayHello("Amit");
```

Streams

A stream is a pipeline for processing data.

Streams are often created from collections:

```
List<String> names = List.of("Ana", "Brian", "Carlos");
```

```
names.stream()
    .forEach(name -> System.out.println(name));
```

A stream does not store data. It processes data from a source, such as a `List` , `Set` , or array.

Common stream operations:

- `filter()`
- `map()`
- `sorted()`
- `distinct()`
- `limit()`
- `forEach()`
- `count()`
- `collect()`
- `reduce()`

Filtering

`filter` keeps only items that match a condition.

```
List<Integer> numbers = List.of(4, 9, 12, 15, 20);

List<Integer> evenNumbers = numbers.stream()
    .filter(n -> n % 2 == 0)
    .toList();

System.out.println(evenNumbers);
```

Output:

```
[4, 12, 20]
```

The lambda:

```
n -> n % 2 == 0
```

means: keep numbers where the remainder after dividing by 2 is zero.

Mapping

`map` transforms each item into something else.

```
List<String> names = List.of("ana", "brian", "carlos");

List<String> upperNames = names.stream()
    .map(name -> name.toUpperCase())
    .toList();
```

```
System.out.println(upperNames);
```

Output:

```
[ANA, BRIAN, CARLOS]
```

You can also use a method reference:

```
List<String> upperNames = names.stream()
    .map(String::toUpperCase)
    .toList();
```

`String::toUpperCase` means: call `toUpperCase` on each `String` .

Filter And Map Together

```
List<String> names = List.of("Alex", "Bo", "Christopher", "Diya", "Eleanor");

List<String> longNamesUpper = names.stream()
    .filter(name -> name.length() > 4)
    .map(String::toUpperCase)
    .toList();

System.out.println(longNamesUpper);
```

Output:

```
[CHRISTOPHER, ELEANOR]
```

Reduce

`reduce` combines many values into one value.

```
List<Integer> numbers = List.of(10, 20, 30);

int total = numbers.stream()
    .reduce(0, (a, b) -> a + b);

System.out.println(total);
```

Output:

```
60
```

Shorter version:

```
int total = numbers.stream()
    .reduce(0, Integer::sum);
```

Collecting Results

Modern Java commonly uses:

```
.toList()
```

Example:

```
List<String> result = names.stream()
    .filter(name -> name.startsWith("A"))
    .toList();
```

You may also see:

```
List<String> result = names.stream()
    .filter(name -> name.startsWith("A"))
    .collect(Collectors.toList());
```

This requires:

```
import java.util.stream.Collectors;
```

Lazy Evaluation

Intermediate operations like `filter` and `map` do not run immediately.

This prints nothing:

```
names.stream()
    .filter(name -> {
        System.out.println("Filtering " + name);
        return name.length() > 3;
    });
```

The stream only runs when there is a terminal operation:

```
long count = names.stream()
    .filter(name -> {
        System.out.println("Filtering " + name);
        return name.length() > 3;
    })
    .count();
```

Common terminal operations include:

- `toList()`
- `forEach()`
- `count()`
- `reduce()`
- `collect()`

Streams Vs Loops

Use loops when:

- The logic is complex and step-by-step
- You are modifying several variables
- You need `break` or `continue`
- A normal loop is easier to read

Use streams when:

- You are filtering, mapping, sorting, grouping, or aggregating
- You want a clear data-processing pipeline
- You are not mutating external state

Good stream example:

```
List<String> activeEmails = users.stream()
    .filter(User::isActive)
    .map(User::getEmail)
    .toList();
```

Less ideal stream example:

```
users.stream()
    .forEach(user -> {
        total++;
        log.add(user.getName());
        user.setActive(false);
    });
```

That kind of side-effect-heavy logic is usually clearer as a loop.

Practice Exercises

Exercise 1: Filter Numbers

Given:

```
List<Integer> numbers = List.of(3, 7, 10, 14, 21, 28);
```

Use streams to create a list containing only even numbers.

Expected result:

```
[10, 14, 28]
```

Exercise 2: Convert Names To Uppercase

Given:

```
List<String> names = List.of("ana", "brian", "carlos", "diya");
```

Use `map` to convert every name to uppercase.

Expected result:

```
[ANA, BRIAN, CARLOS, DIYA]
```

Exercise 3: Filter And Map Together

Given:

```
List<String> names = List.of("Alex", "Bo", "Christopher", "Diya", "Eleanor");
```

Return the uppercase names whose length is greater than 4.

Expected result:

```
[CHRISTOPHER, ELEANOR]
```

Exercise 4: Count Matching Items

Given:

```
List<Integer> scores = List.of(55, 72, 88, 91, 43, 67);
```

Count how many scores are 70 or above.

Expected result:

```
3
```

Exercise 5: Find Total Price

Given:

```
List<Double> prices = List.of(19.99, 5.99, 100.00, 49.50);
```

Use `reduce` or `mapToDouble` to calculate the total.

Expected result:

```
175.48
```

Exercise 6: Product Discount

Given:

```
List<Double> prices = List.of(25.00, 100.00, 10.00, 75.00);
```

Use streams to apply a 10% discount only to prices above 20.

Expected result:

```
[22.5, 90.0, 67.5]
```

Exercise 7: Sort Names

Given:

```
List<String> names = List.of("Zara", "Ana", "Brian", "Carlos");
```

Sort the names alphabetically using streams.

Expected result:

```
[Ana, Brian, Carlos, Zara]
```

Exercise 8: Remove Duplicates

Given:

```
List<String> cities = List.of("Paris", "London", "Paris", "Tokyo", "London");
```

Use `distinct()` to remove duplicates.

Expected result:

```
[Paris, London, Tokyo]
```

Exercise 9: Work With Objects

Create a `Product` class:

```
class Product {  
    private String name;  
    private double price;  
    private String category;
```



```
    // constructor, getters
}
```

Create a list of products and use streams to:

1. Find products above 50
2. Get only their names
3. Return the names as a list

Example result:

[Laptop, Monitor]

Exercise 10: Group Products By Category

Given products with categories like Electronics , Books , and Clothing , use:

```
Collectors.groupingBy()
```

to group products by category.

Expected structure:

```
Map<String, List<Product>>
```

Exercise 11: Average Score

Given:

```
List<Integer> scores = List.of(80, 90, 75, 60, 95);
```

Calculate the average using streams.

Hint:

```
mapToInt()
average()
```

Expected result:

Exercise 12: Find First Match

Given:

```
List<String> emails = List.of(
    "admin@example.com",
    "support@test.com",
```

```
"sales@example.com"  
);
```

Find the first email that ends with `@example.com` .

Hint:

```
filter()  
findFirst()
```

Expected result:

```
admin@example.com
```

Module 6 Lab: Streams And Lambdas

Goal

Build a small Java console program that processes employee data using streams.

Create a file named:

```
Module6StreamsLab.java
```

Starter Code

```
import java.util.Comparator;  
import java.util.List;  
import java.util.Map;  
import java.util.Optional;  
import java.util.stream.Collectors;  
  
class Employee {  
    private String name;  
    private String department;  
    private double salary;  
    private boolean active;  
  
    public Employee(String name, String department, double salary, boolean active) {  
        this.name = name;  
        this.department = department;  
        this.salary = salary;  
        this.active = active;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```

    public String getDepartment() {
        return department;
    }

    public double getSalary() {
        return salary;
    }

    public boolean isActive() {
        return active;
    }

    @Override
    public String toString() {
        return name + " - " + department + " - $" + salary + " - active: " + active;
    }
}

public class Module6StreamsLab {
    public static void main(String[] args) {
        List<Employee> employees = List.of(
            new Employee("Ana", "Engineering", 95000, true),
            new Employee("Brian", "Sales", 72000, true),
            new Employee("Carlos", "Engineering", 105000, false),
            new Employee("Diya", "HR", 68000, true),
            new Employee("Eleanor", "Engineering", 115000, true),
            new Employee("Farah", "Sales", 88000, false),
            new Employee("George", "HR", 76000, true)
        );

        // Complete each task below.
    }
}

```

Task 1: List Active Employees

Use `filter()` to get only active employees.

Expected names:

```

Ana
Brian
Diya
Eleanor
George

```

Task 2: Engineering Employee Names

Use `filter()` and `map()` to get names of employees in Engineering.

Expected result:

```
[Ana, Carlos, Eleanor]
```

Task 3: Count High Earners

Count employees with salary greater than 90000.

Expected result:

```
3
```

Task 4: Average Salary

Calculate the average salary.

Hint:

```
mapToDouble(Employee::getSalary)
average()
```

Expected result:

```
88428.57142857143
```

Task 5: Group By Department

Group employees by department.

Hint:

```
Collectors.groupingBy(Employee::getDepartment)
```

Expected keys:

```
Engineering
Sales
HR
```

Task 6: Highest-Paid Employee

Find the employee with the highest salary.

Hint:

```
max(Comparator.comparingDouble(Employee::getSalary))
```

Expected result:

```
Eleanor
```

Task 7: Salary Raise For Active Employees

Create a new list of strings showing active employees with a 10% salary raise.

Example output:

```
Ana: 104500.0  
Brian: 79200.0  
Diya: 74800.0  
Eleanor: 126500.0  
George: 83600.0
```

Task 8: Any Inactive Engineering Employee?

Use `anyMatch()` to check whether there is an inactive employee in Engineering.

Expected result:

```
true
```

Bonus Challenge

Create a `Map<String, Double>` where:

- key = department name
- value = average salary for that department

Hint:

```
Collectors.groupingBy(  
    Employee::getDepartment,  
    Collectors.averagingDouble(Employee::getSalary)  
)
```

This lab practices the main Module 6 skills: `filter`, `map`, `count`, `average`, `collect`, `groupingBy`, `max`, and `anyMatch`.